

Builders Challenge Reference

A deeper dive into the architecture behind [README.md](#) — the models, the DataEngine pipeline, and the full video corpus. Team setup and credentials are covered there; this doc doesn't repeat them.

Models (NVIDIA on CoreWeave)

Inference for the pipeline runs on shared **GPU endpoints** (NVIDIA Cosmos + YOLO + Canary). You don't deploy the models yourself.

Model	Role in VSS	Endpoint (env var, in <code>/config/<team>.config</code>)
NVIDIA Cosmos3-Reason (<code>nvidia/cosmos3-reason</code>)	Video understanding / reasoning over segments	<code>\$COSMOS3_REASON_URL</code>
YOLO11 (<code>yolo11s</code> , Ultralytics)	Object detection (bounding boxes / labels)	<code>\$YOLO_URL</code>
NVIDIA Cosmos Embed1 (<code>nvidia/cosmos-embed1</code>)	Text and visual embeddings (256-dim) for hybrid search	<code>\$COSMOS_EMBED1_URL</code>
NVIDIA Canary-1B (<code>nvidia/canary-1b</code>)	Speech-to-text (ASR) + speech translation — not Cosmos; NeMo audio model	<code>\$CANARY_1B_URL</code>

Each model has its own host and port; don't assume they share a host. No auth token is needed to call them. Ask Cursor / see `.cursor/skills/gpu/` for how to call each model.

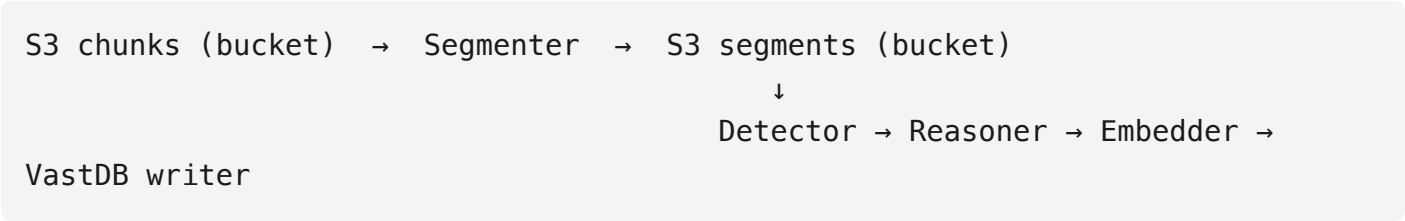
Ingest and search call **Cosmos3-Reason**, **YOLO11**, and **Embed1** through your pipeline and backend today.

Canary-1B is not wired into the current VSS pipeline. If you want audio transcripts, spoken-word search, or speech translation in your demo — let your imagination go and hook it in. **Ask Cursor how to use Canary-1B** against your stack and skills.

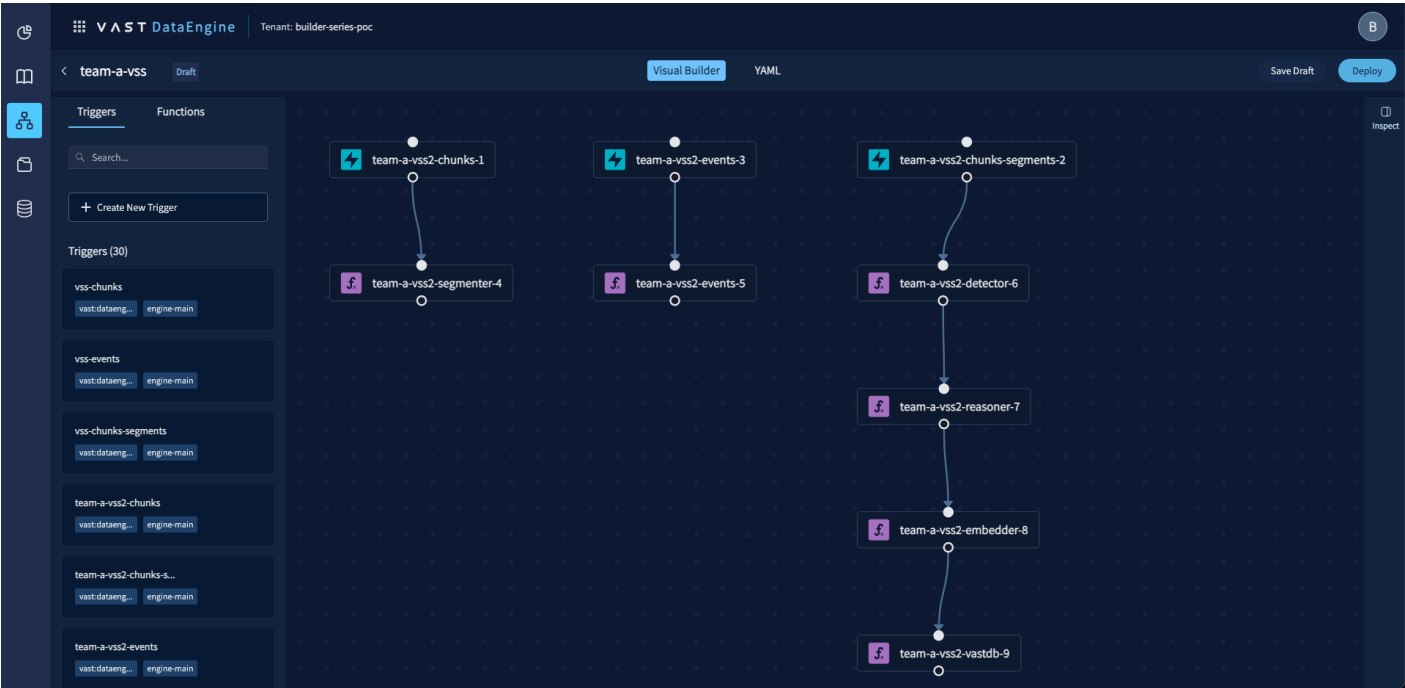
Data Engine

VSS Blueprint

Your team’s ingest runs as a **VAST DataEngine** serverless pipeline. A video chunk lands in S3, then functions run in sequence until searchable rows exist in VastDB.



A separate **events / prompt-suggester** function runs on a schedule and feeds UI suggestions.



VAST DataEngine (pipeline’s tab) — view the pipeline’s flow, logs, and traces. Treat the graph as given for the challenge (re-ingest / upload via skills; don’t rebuild functions or redeploy the pipeline).

Function	What it does
Segmenter	Splits each uploaded chunk into short fixed-length clips and writes them to the segments bucket. Builders challenge: organizers already used the Segmenter to pre-ingest your corpus. During the challenge you only re-ingest data that is already segmented and indexed — the Segmenter is not in the path you run.

Function	What it does
Detector	Runs YOLO11 on each segment and records object classes, counts, and bbox sidecars.
Reasoner	Calls NVIDIA Cosmos3-Reason to write a searchable natural-language description of the segment.
Embedder	Calls NVIDIA Cosmos Embed1 to build text (and visual) vectors for hybrid search.
VastDB writer	Persists embeddings, reasoning, detections, and metadata as a row in your VastDB collection.
Events (prompt-suggester)	Periodically scans recent segments and writes suggested search prompts / key events for the UI.

Builders challenge note: your archive is **pre-ingested** (Segmenter already ran). Your live path is **re-ingest** on existing segments: Detector → Reasoner → Embedder → VastDB writer (via `ingest/reingest-videos` / `reingest-chunk`). You don't upload new chunks or invoke the Segmenter for this challenge.

You don't need to redeploy this graph for the hackathon — treat it as the engine behind search, dashboard, suggestions, and re-ingest.

What you were given

For your team you already have:

Piece	What it is
Ingest pipeline	DataEngine graph that pre-ingested your corpus (Segmenter already ran). During the challenge you re-ingest existing segments (detect → reason → embed → write VastDB)
UI	Web app at your <code>INGRESS_URL</code> (frontend + backend)
S3 buckets	Chunks + segments for uploads
VastDB	Indexed segments, embeddings, detections, reasoning text

Piece	What it is
This repo in Cursor	Agent skills that know how to call every important API — open this project in Cursor and describe what you want
Source code repo	Full VSS Blueprint (<code>vss-blueprint</code>) — optional reading; the live stack is already up

```
cd /vss-blueprint
```

Cloning the Blueprint is **not a requirement**. The live UI, APIs, and Cursor skills in this repo are enough for a strong demo — search, filters, re-ingest, upload, dashboards, mini-apps on top of the archive. Do not rebuild or redeploy DataEngine ingest functions for the challenge.

Open the UI, log in with your team user, and open **this skills repo** in **Cursor**. Skills are how you move fast against the live APIs.

Video corpus (already indexed)

Your team's VSS archive is **pre-ingested** from the lab corpus below. Explore and **re-ingest** with the prompt/metadata you need (`ingest/reingest-videos` / `reingest-chunk`).

One pipeline, one VastDB index, several operational lenses. Only `scenario` + metadata change. Anchor demo line:

Show me every clip, from any camera in any site, where a person is close to a moving vehicle

That should pull I-24 highway traffic, Toronto live driving, neighborhood / SF street cams, and warehouse forklift / aisle views into one result set.

Search UI filters (upload metadata): `camera_id` , `capture_type` , `location` , plus **object class** from detections. Corpus tables below also list a Category column for demo grouping — that is not a filter field. For re-ingest, set analysis `scenario` (presets: `surveillance` , `traffic` , `live_driving` , `retail` , `warehouse` , `egocentric` , `sports` , `nhl` , `general`).

Use-case groups

1. Highway Multi-Cam Traffic — traffic

Source	Location	Category	camera_id	What's indexed
I-24 / 3D traffic	nashville	Traffic	i24_cam-1	~51 multi-cam highway clips (scene*_p*c*)

Multiple cameras per scene. Story: congestion, lane behavior, vehicle–vehicle interaction on a highway corridor. Try: *“truck changing lanes”, “dense traffic on the highway”, “vehicle braking hard.”*

2. Live Driving & Road Safety — live_driving / streets

Source	Location	Category	camera_id	What's indexed
PIE drives	toronto	Streets	pie_cam-3	6 long drive sets (set01 ... set06)

Forward-facing / on-road drives. Story: road-scene understanding and driving safety. Try: *“pedestrian near the curb”, “intersection with turning traffic”, “vehicle ahead braking.”*

3. Neighborhood Street Surveillance — surveillance / streets

Source	Location	Category	camera_id	What's indexed
Neighborhood cars	neighborhood	Streets	neighborhood_cam-1	2 day merges (2026-09-01, 2026-09-02)

Residential / neighborhood car activity. Story: street-level vehicle events over time. Try: *“car passing in front of houses”, “two vehicles close together”, “vehicle stopping at the curb.”*

4. SF Street Surveillance — surveillance / streets

Source	Location	Category	camera_id	What's indexed
SF streets	san_francisco	Streets	sf_streets_cam-1 ... sf_streets_cam-4	4 street cameras (sf-streets/1 ... 4) — ingesting soon

Fixed San Francisco street cams. Story: urban street activity across multiple viewpoints. Try: *“pedestrians crossing while cars wait”, “busy intersection”, “same moment from two SF cameras.”*

5. Warehouse Safety & Operations — warehouse

Source	Location	Category	camera_id	What's indexed
SDG warehouse RGB	warehouse3	Warehouse	sdg_warehouse_cam-2	~178 short ceiling / aisle clips

Industrial **safety & near-miss** search — *“forklift near a person in an aisle”, “pallet in a walkway”, “person in a restricted zone.”*

6. Indoor / Smart-Space Surveillance — surveillance / crowds

Source	Location	Category	camera_id	What's indexed
Smart spaces	indoor	Crowds	smartspace_cam-1	~102 indoor / facility camera clips

Occupancy, people flow, and facility ops. Try: *“person walking through an aisle”, “group of people near equipment”, “empty corridor.”*

Cross-group demos (the payoff)

Query	Hits across groups
<i>“person close to a moving vehicle”</i>	I-24 (1) + Toronto driving (2) + neighborhood (3) + SF streets (4) + warehouse (5)
<i>“dense traffic / many vehicles”</i>	I-24 multi-cam (1) + PIE sets (2) + neighborhood (3) + SF streets (4)
<i>“person in a warehouse aisle”</i>	SDG RGB (5) + smartspaces (6)
<i>“same city, different camera”</i>	SF: sf_streets_cam-1 ... sf_streets_cam-4

Recommended demo packs

Don't try to demo everything at once. Start with a short pack, prove search → filters → UI, then expand.

Pack	Sources	Story
A — Highway Traffic	I-24 multi-cam	Corridor traffic
B — Live Driving	PIE sets 01–06	On-road / road-safety
C — Warehouse Safety	SDG RGB	Near-miss + aisle ops
D — Neighborhood Streets	Neighborhood day 01–02	Day-scale street vehicles
E — SF Streets	SF cams 1–4	Multi-cam urban streets (ingesting soon)
F — Indoor Smart Spaces	Smart spaces cams	Facility / indoor crowds

Start with **Pack A (Highway Traffic)** or **Pack C (Warehouse Safety)** — high-signal — before leaning on the long PIE / Neighborhood day merges. Add **Pack E (SF Streets)** once those cameras are indexed.

Example queries

- “Truck changing lanes on the highway” → i24_cam-1
 - “Pedestrian near the road while driving” → pie_cam-3
 - “Car passing houses on a residential street” → neighborhood_cam-1
 - “Busy San Francisco intersection” → sf_streets_cam-1 ... sf_streets_cam-4
 - “Forklift approaching a person in a warehouse aisle” → sdg_warehouse_cam-2
 - “Person walking through an indoor corridor” → smartspace_cam-1
 - “Person close to a moving vehicle” → cross-pack (A + B + C + D + E)
-

Suggested playbook

Write full prompts in Cursor: name what you want, point at your team config, and say what to build. A solid loop is **re-ingest** → **confirm indexing** → **ship a thin app on search / videos / dashboard**.

Example prompts (adapt to your use case)

1. Re-ingest and verify indexing

Re-ingest an indexed video from Pack A (Highway Traffic) or Pack C (Warehouse Safety) in my team environment (credentials in `/config/<my-team>.config`). Let me pick the target (e.g. an I-24 scene clip or an SDG warehouse clip), prompt/metadata behavior, and chunk count. Wait until re-ingest finishes, then run basic sanity checks on counts and pipeline health. (Cursor will use skills: `ingest/reingest-videos`, `retrieval/dashboard`, `retrieval/login`)

2. Cross-camera “person near vehicle” board

Build a small webpage that answers: “*person close to a moving vehicle*” across my archive. Group hits by `location` / `camera_id` (Nashville I-24, Toronto driving, neighborhood streets, SF streets when indexed, warehouse). For each hit, show a triptych: one clip **before**, the **event**, and one clip **after** (~5 seconds each). Deploy it **on Kubernetes** (not local) to my team namespace without Docker build/push — Ingress path `/app` on my team host. (Cursor will use skills: `retrieval/login`, `retrieval/search`, `retrieval/videos`, `retrieval/list-metadata`, `deployment/deploy-app-no-registry`)

3. Warehouse safety ops board

Build a standalone page for Pack C: histogram of top detected objects + a list of near-miss style hits (forklift near a person, tight aisle, person in a walkway). For each object, show a small bounding-box crop from a real segment. Filter by `camera_id` / `location`. Deploy on K8s at `/app` (or save under `tools/` while iterating), then report the top findings. (Cursor will use skills: `retrieval/login`, `retrieval/dashboard`, `retrieval/search`, `retrieval/videos`, `deployment/deploy-app-no-registry`)

Rules of the road

- Stay in **your** team credentials, buckets, and UI. Don’t poke other teams’ namespaces.
- Prefer **Cursor + skills** over hand-copying curl forever — but reading a skill once to understand the API is encouraged.
- Don’t burn the whole hackathon redeploying infrastructure; the stack is already up.
- If search returns nothing: check login, check dashboard/pipeline alignment, then check that metadata filter values actually exist (`retrieval/list-metadata`).
- Have fun — the win is a crisp story: *problem* → *video archive (one of the packs)* → *search/filters across cameras* → *insight or action*.